

RV-Sparse : Coding challenge

Goal was to implement sparse multiply function

- extracts row major matrix into CSR format for efficient storage
- compute the product with vector
- zero dynamic memory allocation

Implementation

As matrix entries are of type double, need to set tolerance to check for equality with 0, we set tol = 1e-7.

```
const double tol = 1e-7;
```

Key things used to get CSR representation of matrix were:

- matrix entry A[i] is non-zero when fabs(A[i]) > tol
- rows_ptrs always starts with 0, rows_ptrs[i + 1] - rows_ptrs[i] give number of non-zero entries in matrix, hence row_ptrs[i] needs to update how many non-zero numbers encountered so far till row (i - 1)

```
*out_nnz = 0; // number of non-zero elements
row_ptrs[0] = 0;
int idx = 0; // index in values and col_indices array
              // for non-zero values

for (int i = 0; i < rows * cols; ++i) {
    if (fabs(A[i]) > tol) { // non-zero value
        values[idx] = A[i];
        col_indices[idx] = i % cols;
        *out_nnz += 1;
        idx++;
    }

    // update row_ptrs when row ends
    if ((i + 1) % cols == 0) {
        row_ptrs[(i + 1) / cols] = *out_nnz;
    }
}
```

As for multiplication with vector, we need to keep track of where matrix entries are non-zero col-wise, obtained from col_indices to get corresponding multiplier in vector x

```
for (int i = 0; i < rows; ++i) {
    int nums_in_rows = row_ptrs[i + 1] - row_ptrs[i];
    if (nums_in_rows == 0) {
        y[i] = 0;
        continue;
    }
    for (int j = 0; j < nums_in_rows; ++j) {
        int tmp_idx = row_ptrs[i] + j;
        y[i] += values[tmp_idx] * x[col_indices[tmp_idx]];
    }
}
```

Reference

<https://github.com/waternsky/rv-sparse-challenge>